

BÀI GIẢNG: SỬ DỤNG TIMER1 ĐỂ TẠO DELAY TRÊN STM32

1. Giới thiệu

Timer1 là một bộ đếm thời gian 16-bit trên STM32F1, hoạt động ở nhiều chế độ khác nhau:

- **Upcounter:** đếm từ 0 đến giá trị ARR, sau đó tràn về 0.
- **Downcounter:** đếm ngược từ ARR về 0.
- **Center-aligned:** đếm lên và xuống luân phiên.

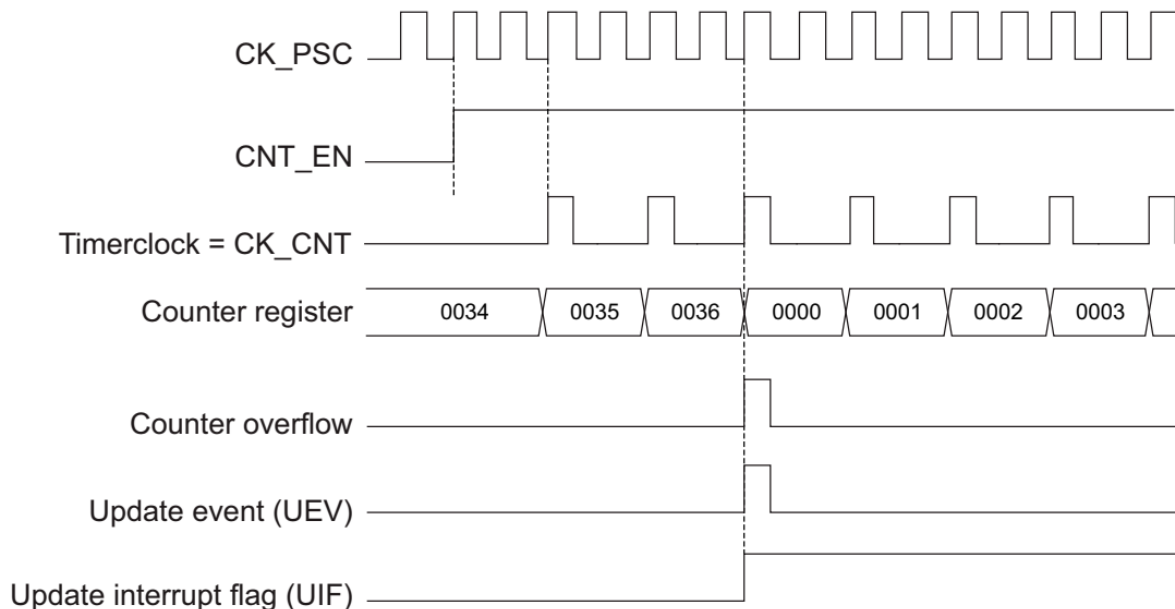
Trong ứng dụng tạo delay, Timer1 thường hoạt động ở chế độ **upcounter**, dựa trên tần số clock đầu vào và cấu hình **Prescaler (PSC)** + **Auto-Reload Register (ARR)** để tạo khoảng thời gian chính xác.

2. Nguyên lý hoạt động của Timer trong việc tạo Delay

2.1. Ý tưởng chung

- Timer nhận xung clock từ hệ thống (f_{clk}).
- Prescaler (PSC) chia nhỏ tần số này để giảm tốc độ đếm.
- Counter (CNT) sẽ tăng mỗi xung clock sau chia.
- Khi $CNT = ARR \rightarrow$ Timer **tràn** $\rightarrow$ cờ UIF (Update Interrupt Flag) bật.

Giản đồ thời gian của TIMER1 đếm lên khi $TIM1_ARR=0x36$, $PSC=2$



2.2. Sơ đồ nguyên lý

f_{clk} (72 MHz) $\rightarrow$ [PSC] $\rightarrow$ Tín hiệu chậm hơn $\rightarrow$ [CNT] $\rightarrow$ So sánh với ARR $\rightarrow$ Tràn (UIF = 1)

2.3. Công thức tính thời gian delay

- Thời gian một lần đếm:

$$T_{\text{tick}} = (\text{PSC} + 1) / f_{\text{clk}}$$

- Thời gian tràn Timer (T_{delay}):

$$T_{\text{delay}} = T_{\text{tick}} \times (\text{ARR} + 1)$$

Ví dụ:

- $f_{\text{clk}} = 72 \text{ MHz}$
 - $\text{PSC} = 7199 \rightarrow T_{\text{tick}} = (\text{PSC} + 1) / f_{\text{clk}} = (7199 + 1) / 72 \text{ MHz} = 100 \mu\text{s}$
- $$\text{ARR} = 9 \rightarrow T_{\text{delay}} = T_{\text{tick}} \times (\text{ARR} + 1) = 100 \mu\text{s} \times (9 + 1) = 1 \text{ ms}$$

3. Các thanh ghi quan trọng của Timer1

Thanh ghi	Chức năng	Mô tả
PSC (Prescaler)	Bộ chia tần số	Chia tần số clock để giảm tốc độ đếm của Timer
ARR (Auto-Reload Register)	Giá trị nạp lại	CNT đếm từ 0 đến ARR rồi tràn
CNT (Counter)	Bộ đếm	Lưu giá trị đếm hiện tại
SR (Status Register)	Trạng thái	Bit UIF = 1 khi Timer tràn
CR1 (Control Register 1)	Điều khiển Timer	Bit CEN bật/tắt Timer, DIR chọn hướng đếm

3.1 Thanh ghi RCC->APB2ENR

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
0	AFIOEN	Clock cho AFIO	0	Không dùng
2	IOPAEN	Clock cho GPIOA	0	Không dùng
3	IOPBEN	Clock cho GPIOB	0	Không dùng

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
4	IOPCEN	Clock cho GPIOC	1	Dùng – cấp clock cho LED (PC13)
11	TIM1EN	Clock cho Timer1	1	Dùng – cấp clock cho Timer1
12	SPI1EN	Clock cho SPI1	0	Không dùng
Các bit khác	...	...	0	Không dùng

3.2 Thanh ghi PSC – TIMx_PSC (Prescaler Register)

Địa chỉ: TIM1->PSC (16 bit, offset 0x28 so với base của TIM1)

Chức năng: Chia tần số clock đầu vào của Timer thành tốc độ đếm mong muốn.

Cấu trúc bit:

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
15:0	PSC	Giá trị bộ chia tần số (Prescaler value)	Ví dụ: 7199	Nếu clock TIM1 là 72 MHz, đặt PSC=7199 → tốc độ đếm = 72 MHz / (7199+1) = 10 kHz (mỗi xung đếm = 0.1 ms)
31:16	Reserved	Không sử dụng		

3.3 Thanh ghi ARR – Auto-Reload Register

Địa chỉ: TIM1->ARR (16 bit, offset 0x2C)

Chức năng: Xác định giá trị đếm tối đa trước khi Timer tràn và reset về 0.

Cấu trúc:

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
15:0	ARR	Giá trị đếm tối đa	9999	Với PSC=7199, ARR=9999

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
				→ thời gian tràn = 1 s
31:16	Reserved	Không sử dụng	-	

Công thức thời gian tràn:

$$T_{\text{overflow}} = (ARR + 1) \times T_{\text{tick}}$$

3.4 Thanh ghi CNT – Counter Register

Địa chỉ: TIM1->CNT (16 bit, offset 0x24)

Chức năng: Lưu giá trị đếm hiện tại của Timer.

Cấu trúc:

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
15:0	CNT	Giá trị bộ đếm hiện tại	Thay đổi liên tục từ 0 đến ARR	Có thể ghi 0 để reset đếm
31:16	Reserved	Không sử dụng		

3.5 Thanh ghi SR – Status Register

Địa chỉ: TIM1->SR (16 bit, offset 0x10)

Chức năng: Lưu trạng thái cờ (flags) của Timer.

Cấu trúc (một số bit quan trọng):

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
0	UIF	Update Interrupt Flag	1 khi Timer tràn	Dùng để kiểm tra hết thời gian delay
1	CC1IF	Capture/Compare 1 Interrupt Flag	0	Không dùng
...	...	...	...	...

3.6. Thanh ghi CR1 – Control Register 1

Địa chỉ: TIM1->CR1 (16 bit, offset 0x00)

Chức năng: Điều khiển hoạt động cơ bản của Timer.

Cấu trúc (một số bit quan trọng):

Bit	Tên bit	Chức năng	Giá trị trong ví dụ	Ghi chú
0	CEN	Counter Enable	1	Bật bộ đếm Timer
1	UDIS	Update Disable	0	Cho phép update
2	URS	Update Request Source	0	Reset khi tràn
7	ARPE	Auto-Reload Preload Enable	0	Không preload ARR

4. Trình tự tạo delay bằng Timer

1. **Bật clock** cho Timer1.
2. **Cấu hình PSC** để giảm tần số đếm.
3. **Cấu hình ARR** để đạt thời gian mong muốn.
4. **Reset CNT và UIF** để bắt đầu mới.
5. **Bật Timer** bằng CEN = 1.
6. **Chờ UIF = 1** để kết thúc delay.
7. **Tắt Timer** nếu không dùng nữa.

5. Ví dụ code tạo delay bằng Timer1

```
#include "stm32f10x.h"
```

```
// Hàm delay ms sử dụng Timer1
```

```
void Timer1_Delay_ms(uint16_t ms) {  
    RCC->APB2ENR |= (1 << 11);    // Bật clock cho TIM1  
    TIM1->PSC = 7199;                // Prescaler: 72MHz / (7199+1) = 10kHz  
    TIM1->ARR = ms * 10 - 1;          // 1 tick = 0.1ms → ms*10 ticks = ms delay  
    TIM1->CNT = 0;                    // Reset counter  
    TIM1->SR &= ~(1 << 0);           // Xóa cờ UIF  
    TIM1->CR1 |= (1 << 0);            // Bật TIM1  
}
```

```

while (!(TIM1->SR & (1 << 0))); // Chờ tràn
TIM1->CR1 &= ~(1 << 0);    // Tắt TIM1
}

int main(void) {
    RCC->APB2ENR |= (1 << 4);    // Bật clock GPIOC
    GPIOC->CRH &= ~(0xF << ((13 - 8) * 4));
    GPIOC->CRH |= (0x2 << ((13 - 8) * 4)); // PC13 output 2MHz push-pull
    while (1) {
        GPIOC->ODR ^= (1 << 13); // Đảo trạng thái LED
        Timer1_Delay_ms(500);    // Delay 500ms
    }
}

```